LendWise

LOAN MANAGEMENT SYSTEM

Backend Architecture & API Plan

Python FastAPI | MongoDB | Redis | Celery | S3 | Docker | YAML

Prepared for the backend team | 6 November 2026

11	72	33	4
modules	APIs	reusable APIs	engineers

Module ownership

Soughat	M1 Auth & Profile, M2 System, Config & Audit	**17 APIs**
Shahid	M3 Loan Applications, M4 Documents, M5 KYC & Bank Verification, M6 Notifications	**19 APIs**
Manar	M7 Agent Management, M8 Agent Workflow & Verification	**19 APIs**
Shabazz	M9 Customer Management, M10 Decisions & Disbursement, M11 Dashboards & Reports	**17 APIs**

Scope: architecture, structure and API plan only. Endpoints are not implemented yet.

1. Overview

This document defines the production backend for the LendWise loan management system that powers the existing React frontend (customer, agent and manager portals). It covers the architecture, the folder structure created in **~/Desktop/loan-management-backend**, the Docker and YAML setup, and the full API plan: **72 APIs in 11 modules**, of which **33 are reusable** across roles. Each module has one owner.

The endpoints are **not implemented yet**. Every module already has its router, schemas, service, repository and models files; each router.py lists the endpoints its owner has to build.

Owner	Focus	Modules	APIs
Soughat	Platform lead - core framework, Docker/YAML, CI/CD, auth, config, audit	M1 Auth & Profile, M2 System, Config & Audit	**17**
Shahid	Customer journey - applications, documents, KYC & bank, notifications	M3 Loan Applications, M4 Documents, M5 KYC & Bank Verification, M6 Notifications	**19**
Manar	Agent domain - agent management, geo matching, accept/decline, verification	M7 Agent Management, M8 Agent Workflow & Verification	**19**
Shabazz	Manager domain - customers, approvals, disbursement, dashboards & reports	M9 Customer Management, M10 Decisions & Disbursement, M11 Dashboards & Reports	**17**
Total		**11 modules**	**72**

API methods: **GET** 34, **POST** 28, **PATCH** 7, **DELETE** 2, **PUT** 1.

2. Technology stack

Layer	Choice	Why
API framework	Python 3.12, FastAPI, Pydantic v2	Async, typed, automatic OpenAPI docs
Database	MongoDB 7 (replica set), Motor + Beanie ODM	Flexible application document, 2dsphere geo queries, transactions
Cache / broker	Redis 7	Caching, rate limits, sessions, Celery broker
Background jobs	Celery workers + Celery beat	Notifications, credit pull, virus scan, payouts, exports
File storage	AWS S3 (MinIO locally)	KYC documents via presigned URLs, encrypted at rest
Gateway	Nginx	TLS, rate limiting, security headers, load balancing
Containers	Docker, Docker Compose (YAML)	Same image for dev, CI, staging, production
Configuration	YAML files + environment variables	base.yaml -> {env}.yaml -> env vars; secrets only in env
CI/CD	GitHub Actions (YAML)	Lint, type-check, tests, build and push image
Observability	structlog JSON logs, Prometheus, Sentry	Request IDs, metrics, error tracking
Auth	JWT (access 15 min, refresh 7 days), argon2	Role-based access: customer, agent, manager

3. System architecture

All traffic enters through Nginx. The FastAPI service is stateless, so it scales horizontally; anything slow or involving a third party runs in Celery workers. MongoDB is the system of record, Redis handles caching, rate limits and the job queue, and documents live in S3.

Figure 1. Runtime architecture

Request flow

- The React app calls /api/v1 with a Bearer access token.
- Nginx terminates TLS, applies rate limits and forwards to an API replica with a request ID.
- The router validates input (Pydantic), require_roles() checks the role, the service applies business rules and the state machine, the repository performs an atomic MongoDB update.
- The service records history + audit and publishes a domain event; Celery sends SMS / e-mail / in-app notifications.
- The response uses a role-specific schema (masked fields, limited view for agents before accepting).

4. Code architecture

Modular monolith: one deployable service, split into feature modules with strict layers. Modules talk to each other through services and domain events, never through each other's repositories. This keeps the four owners independent and lets a module be split into its own service later if needed.

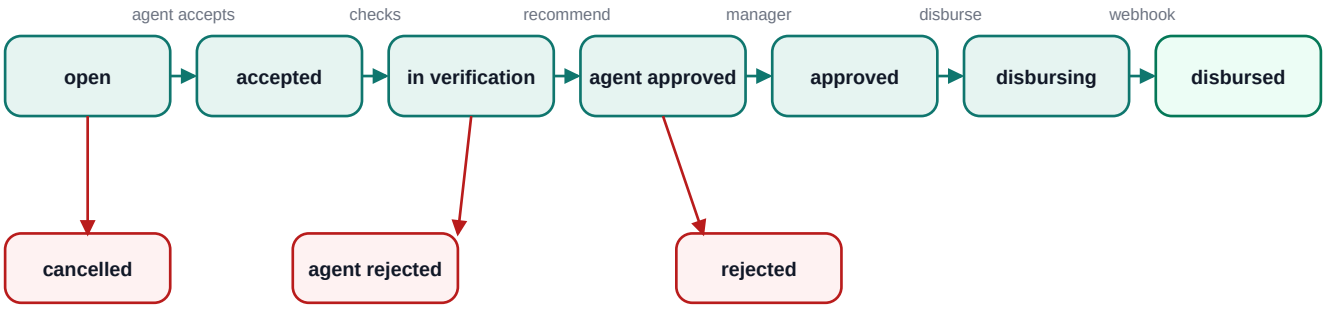
Layer	Location	Responsibility
Router	app/modules/<m>/router.py	HTTP only: parse request, call service, return schema. No business logic.
Schemas	app/modules/<m>/schemas.py	Pydantic request / response models (DTOs); role-specific response models.
Service	app/modules/<m>/service.py	Business rules, permission checks, state transitions, emits domain events.
Repository	app/modules/<m>/repository.py	MongoDB queries only. Atomic updates and aggregation pipelines live here.
Models	app/modules/<m>/models.py	Beanie documents and embedded models with indexes.
Shared	app/shared/, app/core/	Reusable: RBAC deps, errors, pagination, audit, state machine, geo, masking, EMI.
Integrations	app/integrations/	Adapters for S3, SMS, e-mail, credit bureau, KYC provider, payouts, maps.
Workers	app/workers/	Celery tasks triggered by domain events or schedules.

Folder structure

```
loan-management-backend/
|-- app/
|   |-- main.py                # app factory, router registration
|   |-- core/                  # config, security, deps (RBAC), errors, middleware,
|   |                          #   pagination, idempotency, logging
|   |-- db/                    # mongo.py (Motor/Beanie), indexes.py
|   |-- shared/                # state_machine, geo, masking, emi, audit, events, validators
|   |-- integrations/          # storage (S3), sms, email, credit_bureau, kyc_provider, payouts, maps
|   |-- workers/               # celery_app.py, tasks/ (notify, documents, credit, payouts, reports)
|   `-- modules/               # one package per module, each with:
|       |-- auth/              #   router.py  schemas.py  service.py  repository.py  models.py
|       |-- system/  applications/  documents/  kyc/  notifications/
|       `-- agents/  agent_workflow/  customers/  decisions/  reports/
|-- config/                    # base.yaml, development.yaml, production.yaml, logging.yaml
|-- docker/                    # nginx/nginx.conf
|-- tests/                     # unit/, integration/, conftest.py
|-- docs/                      # ARCHITECTURE.md, api-catalog.yaml
|-- .github/workflows/ci.yml
|-- Dockerfile  docker-compose.yml  docker-compose.prod.yml
`-- requirements.txt  requirements-dev.txt  pyproject.toml  Makefile  .env.example
```

5. Loan application lifecycle

Status changes are only allowed through the state machine in app/shared/state_machine.py. Each transition names the role that may trigger it and its guards, so the rules shown in the frontend are enforced on the server.



Deactivating an agent returns accepted / in-verification cases to "open". Payout failure returns "disbursing" to "approved".

Figure 2. Application status flow

From	To	Who	API	Guard
open	accepted	Agent	`POST /applications/{id}/accept`	Agent active, within 2 km, not declined; atomic
open	cancelled	Customer	`POST /applications/{id}/cancel`	Owner only
open	rejected	Manager	`POST /applications/{id}/reject`	Reason required
accepted	in_verification	Agent	`verification/start, checks, credit-check`	Owning agent only
accepted / in_verification	open	System	`PATCH /agents/{id}/status (deactivate)`	Releases case to the area pool
in_verification	agent_approved	Agent	`POST /applications/{id}/recommend`	Credit >= 650, all checks done, docs present
accepted / in_verification	agent_rejected	Agent	`POST /applications/{id}/agent-reject`	Reason required
agent_approved	approved	Manager	`POST /applications/{id}/approve`	Credit >= 650
agent_approved	rejected	Manager	`POST /applications/{id}/reject`	Reason required
approved	disbursing	Manager	`POST /applications/{id}/disburse`	Idempotency-Key; verified bank account

From	To	Who	API	Guard
disbursing	disbursed / approved	Provider	`POST /webhooks/payouts`	HMAC verified; failure returns to approved

6. Data model (MongoDB)

The application is one rich document (applicant, KYC, bank, verification, decisions embedded) because it is always read and updated as a unit; high-volume or append-only data (events, documents, notifications, audit) lives in separate collections. Sensitive numbers are stored encrypted.

Collection	Key fields	Indexes
users	role, name, email, mobile, password_hash, status; agent: {employee_code, area, location (GeoJSON), active}	email, mobile (unique); location 2dsphere
applications	app_no, customer_id, agent_id, status, loan {amount, tenure, purpose, rate, emi}, applicant, kyc {aadhaar_enc, aadhaar_last4, pan_enc}, bank {holder, account_enc, last4, ifsc}, address, location, declined_by[], verification {checks, comments}, credit_check, decisions, version	status+created_at, agent_id+status, customer_id; location 2dsphere; app_no unique
documents	application_id, owner_id, type, s3_key, mime, size, sha256, scan_status	application_id+type
application_events	application_id, at, actor {id, role, name}, action, from_status, to_status, note	application_id+at (append-only)
disbursements	application_id, amount, mode, account_snapshot (masked), provider_ref, utr, status, attempts	application_id unique; utr unique
notifications	user_id, type, title, body, link, read, created_at	user_id+read+created_at
audit_logs	actor, action, entity, entity_id, before/after diff, ip, request_id	entity+entity_id, actor_id
config	versioned business rules (radius, min credit score, rate, tenures...)	version
sessions / otp_codes / idempotency_keys	hashed refresh tokens, OTP hashes, idempotent responses	TTL indexes (auto-expire)
counters	sequence for human-readable app numbers (LN-1020)	_id

7. Key production design decisions

First agent to accept wins

POST /accept runs one atomic findOneAndUpdate with the filter {_id, status: "open", agent_id: null, "declined_by.agent_id": {$ne: me}} and sets agent_id + status. If it returns nothing, respond 409 ALREADY_ACCEPTED. No locks and no race conditions, even with many API replicas.

2 km area matching

Agents and applications store GeoJSON points with 2dsphere indexes. Open requests for an agent use $geoWithin/$centerSphere with the configured radius, so changing the radius needs no code change.

Role-aware responses

One endpoint, several response schemas. Before accepting, an agent receives only amount, address and location. Aadhaar, PAN and account numbers are always masked except for the owning agent and the manager.

State machine in one place

app/shared/state_machine.py defines every allowed transition, the role that may trigger it and its guards. Services call transition(...); repositories update with the expected current status and version (optimistic locking).

Documents never pass through the API

The client asks for a presigned S3 URL, uploads directly, then confirms. Files are virus-scanned by a worker, stored with SSE-KMS encryption and downloaded through 5-minute presigned URLs.

Idempotent money and submit operations

POST /applications and POST /disburse require an Idempotency-Key header, so retries and double clicks never create duplicate applications or duplicate payouts.

Slow work is asynchronous

Credit bureau pulls, SMS / e-mail, virus scans, payouts and exports run in Celery with retries and backoff. The API returns quickly and the UI updates from status and notifications.

Every change is audited

Each state change writes an application_events record (shown as History in the UI) and an audit_logs record with actor, IP and request ID.

8. Security and compliance

- JWT access tokens (15 min) + rotating refresh tokens (7 days) stored hashed; reuse of an old refresh token revokes the session.
- Passwords hashed with argon2; account lockout after 5 failed logins; OTP attempts limited.
- RBAC through one reusable dependency, require_roles(...), plus ownership checks in services (customer owns the application, agent owns the accepted case).
- Aadhaar, PAN and bank account numbers are encrypted at field level (AES-GCM, key from KMS); only last-4 and an HMAC blind index are stored in clear for search. Follow UIDAI rules for an Aadhaar Data Vault before going live.
- Rate limits at Nginx and in the app (login 10/min, OTP 3/min); request size limited to 1 MB.
- Payout webhooks verified with HMAC signatures and replay protection.
- PII never written to logs (masking filter); CORS restricted to the frontend origin; security headers and TLS 1.2+.
- Dependency and image scanning in CI; secrets only in environment variables or a secret manager, never in YAML or Git.

9. Observability and reliability

- Structured JSON logs with request_id, user_id and route on every line.
- Prometheus metrics: latency, error rate per endpoint, queue length, payout success rate.
- Sentry for exceptions in API and workers.
- Health probes: /health (liveness) and /health/ready (Mongo, Redis, S3).
- MongoDB replica set (Atlas in production) with daily backups and point-in-time recovery.
- API is stateless and runs as 2+ replicas behind Nginx; workers scale independently.

10. Docker, YAML and environments

File	Purpose
`Dockerfile`	Multi-stage build, Python 3.12 slim, non-root user, health check
`docker-compose.yml`	Dev stack: api (hot reload), worker, beat, MongoDB replica set, Redis, MinIO, Mailpit
`docker-compose.prod.yml`	Production: api x2 (gunicorn + uvicorn workers), workers, beat, Redis, Nginx; MongoDB Atlas + AWS S3
`config/base.yaml`	Defaults: business rules (2 km, 650, 50% FOIR, 12%), auth TTLs, documents, rate limits
`config/development.yaml / production.yaml`	Per-environment overrides (docs on/off, JSON logs, Aadhaar demo mode, sandbox)
`config/logging.yaml`	Logging configuration
`docker/nginx/nginx.conf`	TLS, HTTP->HTTPS redirect, rate limit zones, security headers, proxy to API
`.github/workflows/ci.yml`	Ruff, mypy, pytest with Mongo/Redis services, build and push image on main
`.env.example`	All secrets and connection strings (copy to .env)

Environments

Environment	Database	Files	Notes
Development	Mongo container (replica set)	MinIO	docker compose up; hot reload; Mailpit; sandbox integrations
CI	Mongo service container	n/a	GitHub Actions; tests must pass before image build
Staging	MongoDB Atlas (separate cluster)	S3 bucket	Same image as production; sandbox payouts
Production	MongoDB Atlas (replica set, backups)	S3 + SSE-KMS	docker-compose.prod.yml or Kubernetes; docs disabled publicly

Run locally: `cp .env.example .env && docker compose up --build`, then open http://localhost:8000/docs. Configuration loads `config/base.yaml`, then `config/{APP_ENV}.yaml`, then environment variables (env wins). Secrets are only in env.

11. API catalog by module

Base path `/api/v1`. **R** = reusable: one endpoint serves several roles and scopes its response by role. 72 APIs in total.

M1 Auth & Profile Owner: Soughat | 11 APIs

Sign-up, login (email or mobile), JWT access/refresh rotation, OTP, password reset, own profile.

#	Method	Endpoint	Roles	R	Purpose
1	POST	`/auth/register`	Public		Customer sign-up (name, email, mobile, password, terms)
2	POST	`/auth/login`	Public	R	Login with email or mobile; returns tokens + role for redirect
3	POST	`/auth/refresh`	Public	R	Rotate access/refresh tokens (refresh-token reuse detection)
4	POST	`/auth/logout`	Any	R	Revoke current refresh token / session
5	POST	`/auth/otp/send`	Public	R	Send OTP to mobile (sign-up verification, reset)
6	POST	`/auth/otp/verify`	Public	R	Verify OTP and return a short-lived verification token
7	POST	`/auth/forgot-password`	Public	R	Start password reset (same response whether or not user exists)
8	POST	`/auth/reset-password`	Public	R	Set new password using reset token
9	POST	`/auth/change-password`	Any	R	Change password when logged in
10	GET	`/auth/me`	Any	R	Current user, role and permissions
11	PATCH	`/auth/me`	Any	R	Update own profile (mobile change needs OTP)

M2 System, Config & Audit Owner: Soughat | 6 APIs

Health probes, business-rule configuration, EMI calculator, audit trail.

#	Method	Endpoint	Roles	R	Purpose
12	GET	`/health`	Public	R	Liveness probe for Docker / load balancer
13	GET	`/health/ready`	Internal	R	Readiness - checks MongoDB, Redis and object storage
14	GET	`/config`	Any	R	Master data - loan purposes, employment types, tenures, rate, radius, min credit score
15	PUT	`/config`	Manager		Update business rules (versioned, audited)
16	GET	`/config/emi`	Any	R	EMI calculator (amount, tenure) - used by apply form and agent view
17	GET	`/audit-logs`	Manager		Search audit trail by entity, actor and date range

M3 Loan Applications Owner: Shahid | 6 APIs

Submit, list and track applications; role-aware responses; timeline and history.

#	Method	Endpoint	Roles	R	Purpose
18	POST	`/applications`	Customer		Submit application (KYC, loan, tenure, address, GPS, bank, document ids); Idempotency-Key
19	GET	`/applications`	Customer, Agent, Manager	R	Role-scoped list with filters, search and pagination
20	GET	`/applications/{id}`	Customer, Agent, Manager	R	Role-aware detail (agent sees amount/address/location only before accepting)
21	GET	`/applications/{id}/timeline`	Customer, Agent, Manager	R	6-step progress (submitted -> disbursed)
22	GET	`/applications/{id}/history`	Customer, Agent, Manager	R	Event log of every action
23	POST	`/applications/{id}/cancel`	Customer		Withdraw before an agent accepts

M4 Documents Owner: Shahid | 5 APIs

Direct-to-S3 uploads with presigned URLs, metadata, virus scan, secure download.

#	Method	Endpoint	Roles	R	Purpose
24	POST	`/documents/upload-url`	Customer	R	Presigned PUT URL (type, size and MIME validated)
25	POST	`/documents`	Customer	R	Confirm upload and save metadata; queues virus scan
26	GET	`/applications/{id}/documents`	Customer, Agent, Manager	R	List documents of an application
27	GET	`/documents/{id}/download-url`	Customer, Agent, Manager	R	Short-lived presigned GET URL
28	DELETE	`/documents/{id}`	Customer		Remove a document before submission

M5 KYC & Bank Verification Owner: Shahid | 4 APIs

Aadhaar, PAN, IFSC and bank-account validation through providers.

#	Method	Endpoint	Roles	R	Purpose
29	POST	`/kyc/aadhaar/validate`	Customer	R	Format + Verhoeff checksum; optional OTP e-KYC via provider
30	POST	`/kyc/pan/validate`	Customer	R	Format + PAN status/name match via provider
31	GET	`/kyc/ifsc/{ifsc}`	Any	R	Bank and branch lookup (cached in Redis)
32	POST	`/kyc/bank/verify`	Customer, Manager	R	Penny-drop account verification and holder-name match

M6 Notifications Owner: Shahid | 4 APIs

In-app inbox; SMS/e-mail/push are sent by background workers on domain events.

#	Method	Endpoint	Roles	R	Purpose
33	GET	`/notifications`	Any	R	Paginated notification inbox
34	GET	`/notifications/unread-count`	Any	R	Badge count for the bell icon
35	PATCH	`/notifications/{id}/read`	Any	R	Mark one as read
36	PATCH	`/notifications/read-all`	Any	R	Mark all as read

M7 Agent Management Owner: Manar | 9 APIs

Manager CRUD for agents including GeoJSON location; geo search of nearby agents.

#	Method	Endpoint	Roles	R	Purpose
37	POST	/agents	Manager		Create agent with location (lat/lng), area and credentials
38	GET	/agents	Manager		List / search / filter (active, area)
39	GET	/agents/{id}	Manager		Agent details
40	PATCH	/agents/{id}	Manager		Edit agent incl. location
41	PATCH	/agents/{id}/status	Manager		Activate / deactivate; releases unfinished cases to the open pool
42	DELETE	/agents/{id}	Manager		Soft delete (blocked if applications exist)
43	GET	/agents/{id}/applications	Manager		Customers / applications handled by the agent
44	GET	/agents/{id}/performance	Manager		Processed, recommendation rate, manager agreement
45	GET	/agents/nearby	Customer, Manager	R	Active agents within radius of lat/lng (2dsphere)

M8 Agent Workflow & Verification Owner: Manar | 10 APIs

Area request pool, atomic first-to-accept, decline, credit check, field/document checks, recommendation.

#	Method	Endpoint	Roles	R	Purpose
46	GET	/agent/open-requests	Agent		Open requests within 2 km of the agent, excluding declined
47	POST	/applications/{id}/accept	Agent		Atomic claim - first agent wins (findOneAndUpdate)
48	POST	/applications/{id}/decline	Agent		Hide request for this agent (optional reason)
49	POST	/applications/{id}/verification/start	Agent		Move to in-verification
50	PATCH	/applications/{id}/verification/checks	Agent		Tick field and document checks
51	POST	/applications/{id}/credit-check	Agent		Credit bureau pull once (async); stores score
52	POST	/applications/{id}/comments	Agent		Add verification comment
53	GET	/applications/{id}/comments	Agent, Manager	R	List verification comments
54	POST	/applications/{id}/recommend	Agent		Recommend to manager (credit >= 650 and all checks done)
55	POST	/applications/{id}/agent-reject	Agent		Reject with mandatory reason

M9 Customer Management Owner: Shabazz | 4 APIs

Manager views of customers, their application history, block/unblock.

#	Method	Endpoint	Roles	R	Purpose
56	GET	/customers	Manager		Search / filter / paginate customers
57	GET	/customers/{id}	Manager		Customer profile and stats
58	GET	/customers/{id}/applications	Manager		Customer application history
59	PATCH	/customers/{id}/status	Manager		Block / unblock customer

M10 Decisions & Disbursement Owner: Shabazz | 6 APIs

Final approval/rejection and payout to the verified bank account.

#	Method	Endpoint	Roles	R	Purpose
60	POST	/applications/{id}/approve	Manager		Final approval (only after agent recommendation and credit >= 650)
61	POST	/applications/{id}/reject	Manager		Final rejection with mandatory reason
62	POST	/applications/{id}/disburse	Manager		Payout via IMPS/NEFT/RTGS; Idempotency-Key; returns UTR
63	GET	/disbursements	Manager		List payouts with status and filters
64	GET	/disbursements/{id}	Customer, Manager	R	Disbursement receipt
65	POST	/webhooks/payouts	Provider		HMAC-signed payout status callback (UTR, success/failure)

M11 Dashboards & Reports Owner: Shabazz | 7 APIs

Role dashboards (aggregation pipelines, Redis cached) and exports.

#	Method	Endpoint	Roles	R	Purpose
66	GET	/dashboard/customer	Customer		Customer stat cards and latest application
67	GET	/dashboard/agent	Agent		Agent stat cards and to-do list
68	GET	/dashboard/manager	Manager		Manager stat cards and action alerts
69	GET	/reports/applications/monthly	Manager		Monthly applications chart
70	GET	/reports/applications/outcomes	Manager		Approved vs rejected vs in-progress
71	GET	/reports/agents/performance	Manager		Agent performance chart
72	GET	/reports/export	Manager		CSV/XLSX export (async job, download link)

12. Reusable APIs and shared components

33 of 72 endpoints are reusable. Instead of separate customer, agent and manager versions, one endpoint uses the caller's role to scope the query and pick the response schema. Examples: GET /applications, GET /applications/{id}, timeline, history, documents, notifications, /config, /config/emi, IFSC lookup and /agents/nearby.

Reusable endpoint	Roles
POST /auth/login	Public
POST /auth/refresh	Public
POST /auth/logout	Any
POST /auth/otp/send	Public
POST /auth/otp/verify	Public
POST /auth/forgot-password	Public
POST /auth/reset-password	Public
POST /auth/change-password	Any
GET /auth/me	Any
PATCH /auth/me	Any
GET /health	Public
GET /health/ready	Internal
GET /config	Any

Reusable endpoint	Roles
`GET /config/emi`	Any
`GET /applications`	Customer, Agent, Manager
`GET /applications/{id}`	Customer, Agent, Manager
`GET /applications/{id}/timeline`	Customer, Agent, Manager
`GET /applications/{id}/history`	Customer, Agent, Manager
`POST /documents/upload-url`	Customer
`POST /documents`	Customer
`GET /applications/{id}/documents`	Customer, Agent, Manager
`GET /documents/{id}/download-url`	Customer, Agent, Manager
`POST /kyc/aadhaar/validate`	Customer
`POST /kyc/pan/validate`	Customer
`GET /kyc/ifsc/{ifsc}`	Any
`POST /kyc/bank/verify`	Customer, Manager
`GET /notifications`	Any
`GET /notifications/unread-count`	Any
`PATCH /notifications/{id}/read`	Any
`PATCH /notifications/read-all`	Any
`GET /agents/nearby`	Customer, Manager
`GET /applications/{id}/comments`	Agent, Manager
`GET /disbursements/{id}`	Customer, Manager

Shared code components (owned by Soughat unless noted)

Component	Reused by
`require_roles(*roles)`	One RBAC dependency for all 72 endpoints
`Page[T] pagination + filter params`	Same list contract for applications, customers, agents, notifications...
`Error envelope + exception handlers`	Consistent error JSON with code and request_id
`state_machine.transition()`	Single place for status rules used by M3, M8, M10
`audit.record()`	History + audit for every module
`geo.within_radius()`	Used by open requests, nearby agents, submit preview
`masking.*`	Aadhaar / PAN / account masking in responses and logs
`emi.calculate()`	Apply form, agent view, reports
`storage.presign_*()`	Documents and report exports
`Idempotency-Key middleware`	Submit and disburse

13. Team assignment

Soughat - 17 APIs

Platform lead - core framework, Docker/YAML, CI/CD, auth, config, audit

Module	APIs	Endpoints
M1 Auth & Profile	11	`POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout, POST /auth/otp/send, POST /auth/otp/verify, POST /auth/forgot-password, POST /auth/reset-password, POST /auth/change-password, GET /auth/me, PATCH /auth/me`
M2 System, Config & Audit	6	`GET /health, GET /health/ready, GET /config, PUT /config, GET /config/emi, GET /audit-logs`

Shahid - 19 APIs

Customer journey - applications, documents, KYC & bank, notifications

Module	APIs	Endpoints
M3 Loan Applications	6	`POST /applications, GET /applications, GET /applications/{id}, GET /applications/{id}/timeline, GET /applications/{id}/history, POST /applications/{id}/cancel`
M4 Documents	5	`POST /documents/upload-url, POST /documents, GET /applications/{id}/documents, GET /documents/{id}/download-url, DELETE /documents/{id}`
M5 KYC & Bank Verification	4	`POST /kyc/aadhaar/validate, POST /kyc/pan/validate, GET /kyc/ifsc/{ifsc}, POST /kyc/bank/verify`
M6 Notifications	4	`GET /notifications, GET /notifications/unread-count, PATCH /notifications/{id}/read, PATCH /notifications/read-all`

Manar - 19 APIs

Agent domain - agent management, geo matching, accept/decline, verification

Module	APIs	Endpoints
M7 Agent Management	9	`POST /agents, GET /agents, GET /agents/{id}, PATCH /agents/{id}, PATCH /agents/{id}/status, DELETE /agents/{id}, GET /agents/{id}/applications, GET /agents/{id}/performance, GET /agents/nearby`
M8 Agent Workflow & Verification	10	`GET /agent/open-requests, POST /applications/{id}/accept, POST /applications/{id}/decline, POST /applications/{id}/verification/start, PATCH /applications/{id}/verification/checks, POST /applications/{id}/credit-check, POST /applications/{id}/comments, GET /applications/{id}/comments, POST /applications/{id}/recommend, POST /applications/{id}/agent-reject`

Shabazz - 17 APIs

Manager domain - customers, approvals, disbursement, dashboards & reports

Module	APIs	Endpoints
M9 Customer Management	4	`GET /customers, GET /customers/{id}, GET /customers/{id}/applications, PATCH /customers/{id}/status`
M10 Decisions & Disbursement	6	`POST /applications/{id}/approve, POST /applications/{id}/reject, POST /applications/{id}/disburse, GET /disbursements, GET /disbursements/{id}, POST /webhooks/payouts`
M11 Dashboards & Reports	7	`GET /dashboard/customer, GET /dashboard/agent, GET /dashboard/manager, GET /reports/applications/monthly, GET /reports/applications/outcomes, GET /reports/agents/performance, GET /reports/export`

Dependencies between members

- Everyone depends on Soughat's core: auth dependency, error format, pagination, audit and state machine (deliver in week 1).
- Shahid owns the application document schema; Manar (accept/verify) and Shabazz (approve/disburse) build on it.
- Manar's recommend sets agent_approved, which Shabazz's approve requires; agree the guard rules together.
- Shabazz's dashboards aggregate data from all modules; they start with fixtures and switch to real data in week 4.

Suggested plan (6 weeks)

When	Phase	Work
Week 1	Foundation	Soughat: core (config, security, RBAC deps, errors, pagination, audit, state machine), Docker, CI, auth login/register/me. Others: models, schemas and repository for their modules; agree the OpenAPI contract with the frontend.
Week 2-3	Module build	All four build their endpoints in parallel against the shared core, with unit tests for services.
Week 4	Integrations	Sandbox adapters: credit bureau (Manar), KYC + S3 + SMS/e-mail (Shahid), payouts + webhooks (Shabazz), workers (Soughat).
Week 5	Frontend integration	Replace the React localStorage store with API calls; end-to-end tests of the full loan flow.
Week 6	Hardening	Load test (accept race, dashboards), security review, staging deploy, runbooks.

Definition of done (every endpoint)

- Endpoint matches the catalog (method, path, roles) and appears in OpenAPI with examples.
- Uses require_roles, the error envelope and pagination from core; no duplicated helpers.
- Unit tests for service rules and an integration test per endpoint; coverage above 80% for the module.
- State changes go through the state machine and write audit + history.
- No PII in logs; sensitive fields masked in responses.
- Reviewed by one other team member (pairs: Soughat <-> Shahid, Manar <-> Shabazz).